Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Chapter 8
Solving sparse linear systems

Solving a sparse linear system $Ax = b$ is now a matter of putting together the methods described in Chapters 2 through 7. The typical steps are to (1) find a permutation to reduce fill-in, (2) analyze and factorize the permuted matrix A (via Cholesky, LU, or QR), (3) permute the right-hand side b, (4) solve for x using forward/backsolve or by applying Householder reflections, (5) permute the solution x, and (6) optionally perform one or two steps of iterative refinement, which can sometimes improve the solution (see Problem 8.5), even when performed in standard floating-point precision.

A naive method for solving $Ax = b$ when A is square and nonsingular is to compute the inverse A^{-1}, or `x=inv(A)*b` in MATLAB. This is numerically unstable when A is ill-conditioned in the dense case and also very costly in the sparse case. The inverse normally has no zero entries at all, as shown by the following two theorems.

Theorem 8.1 (Gilbert [101])**.** *Ignoring numerical cancellation, the nonzero pattern of the solution to $Ax = b$, where A has a zero-free diagonal, is $\mathcal{X} = Reach_A(\mathcal{B})$. Ignoring numerical cancellation, the solution x has no zero entries if A is strong Hall.*

Theorem 8.2 (Gilbert [101])**.** *The transitive closure of the directed graph of A is the graph C, where $\mathcal{C}_i = Reach_A(i)$. Ignoring numerical cancellation, $\mathcal{C}$ gives the nonzero pattern of A^{-1}. Every edge is present in C, and A^{-1} has no zero entries, if A is strong Hall.*

8.1 Using a Cholesky factorization

When A is symmetric positive definite, the system $Ax = b$ can be solved via Cholesky factorization. If P is the fill-reducing permutation, $LL^T = PAP^T$. The system $Ax = b$ becomes $PAP^TPx = Pb$. Solving $Ly = Pb$ for y, solving $L^Tz = y$ for z, and finally $x = P^Tz$ results in the solution x. In MATLAB notation,

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
    p = amd (A) ;
    L = chol (A) ;
    x = L' \ (L \ b (p)) ;
    x (p) = x ;
```

The `cs_cholsol` function overwrites its input `b` with the solution to $Ax = b$. `order` determines the input ordering used (`0` for $P = I$, or `1` for a minimum degree ordering of A). It returns true (`1`) if successful, false (`0`) if the matrix is not positive definite or if the method ran out of memory. Note that the forward/backsolve steps cannot fail because they do not allocate memory.

```
int cs_cholsol (int order, const cs *A, double *b)
{
    double *x ;
    css *S ;
    csn *N ;
    int n, ok ;
    if (!CS_CSC (A) || !b) return (0) ;     /* check inputs */
    n = A->n ;
    S = cs_schol (order, A) ;               /* ordering and symbolic analysis */
    N = cs_chol (A, S) ;                    /* numeric Cholesky factorization */
    x = cs_malloc (n, sizeof (double)) ;    /* get workspace */
    ok = (S && N && x) ;
    if (ok)
    {
        cs_ipvec (S->pinv, b, x, n) ;   /* x = P*b */
        cs_lsolve (N->L, x) ;           /* x = L\x */
        cs_ltsolve (N->L, x) ;          /* x = L'\x */
        cs_pvec (S->pinv, x, b, n) ;    /* b = P'*x */
    }
    cs_free (x) ;
    cs_sfree (S) ;
    cs_nfree (N) ;
    return (ok) ;
}
```

8.2 Using a QR factorization

The *least squares* problem is to find x that minimizes the 2-norm of the residual, $\|r\|_2$, where $r = b - Ax$ and A is m-by-n with $m \geq n$. Multiplying a vector by an orthogonal matrix Q does not change its 2-norm. If A is factorized into the product $A = QR$, then

$$\|r\|_2 = \|b - Ax\|_2 = \|Q^T b - Rx\|_2 = \left\| \begin{bmatrix} Q_1^T b - R_1 x \\ Q_2^T b \end{bmatrix} \right\|_2 = \left\| \begin{bmatrix} r_1 \\ r_2 \end{bmatrix} \right\|_2,$$

where Q is m-by-m, R_1 is n-by-n, and Q_1 is m-by-n. Assuming A has full rank, R_1 is nonsingular and so the upper triangular system $Q_1^T b = R_1 x$ can be solved, which makes $r_1 = 0$ and minimizes $\|r\|_2$.

QR factorization also provides a reliable method for solving underdetermined systems $Ax = b$, where A is m-by-n with $m < n$. Ignoring the fill-reducing ordering, consider the QR factorization $QR = A^T$. The system becomes $R^T Q^T x = b$. If the upper triangular system $R^T y = b$ is solved for y, the solution to $Ax = b$ is $x = Qy$.

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

In `cs_qrsol`, $Q = H_1 H_2 \dots H_n$ is represented implicitly as a product of Householder reflections, and the permuted matrix $PA\overline{Q}$ is factorized instead of A, where $\overline{Q}$ is the fill-reducing column permutation. The right-hand side `b[0...m-1]` is overwritten with the solution `x[0..n-1]` (and thus `b` must be of size `max(m,n)`). Use `order=0` for the natural ordering or `3` for a minimum degree ordering of $A^T A$.

```
int cs_qrsol (int order, const cs *A, double *b)
{
    double *x ;
    css *S ;
    csn *N ;
    cs *AT = NULL ;
    int k, m, n, ok ;
    if (!CS_CSC (A) || !b) return (0) ; /* check inputs */
    n = A->n ;
    m = A->m ;
    if (m >= n)
    {
        S = cs_sqr (order, A, 1) ;          /* ordering and symbolic analysis */
        N = cs_qr (A, S) ;                  /* numeric QR factorization */
        x = cs_calloc (S ? S->m2 : 1, sizeof (double)) ;    /* get workspace */
        ok = (S && N && x) ;
        if (ok)
        {
            cs_ipvec (S->pinv, b, x, m) ;   /* x(0:m-1) = b(p(0:m-1) */
            for (k = 0 ; k < n ; k++)       /* apply Householder refl. to x */
            {
                cs_happly (N->L, k, N->B [k], x) ;
            }
            cs_usolve (N->U, x) ;           /* x = R\x */
            cs_ipvec (S->q, x, b, n) ;      /* b(q(0:n-1)) = x(0:n-1) */
        }
    }
    else
    {
        AT = cs_transpose (A, 1) ;          /* Ax=b is underdetermined */
        S = cs_sqr (order, AT, 1) ;         /* ordering and symbolic analysis */
        N = cs_qr (AT, S) ;                 /* numeric QR factorization of A' */
        x = cs_calloc (S ? S->m2 : 1, sizeof (double)) ;    /* get workspace */
        ok = (AT && S && N && x) ;
        if (ok)
        {
            cs_pvec (S->q, b, x, m) ;       /* x(q(0:m-1)) = b(0:m-1) */
            cs_utsolve (N->U, x) ;          /* x = R'\x */
            for (k = m-1 ; k >= 0 ; k--)    /* apply Householder refl. to x */
            {
                cs_happly (N->L, k, N->B [k], x) ;
            }
            cs_pvec (S->pinv, x, b, n) ;    /* b(0:n-1) = x(p(0:n-1)) */
        }
    }
    cs_free (x) ;
    cs_sfree (S) ;
    cs_nfree (N) ;
    cs_spfree (AT) ;
    return (ok) ;
}
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

8.3 Using an LU factorization

When A is a general square matrix, solving $Ax = b$ is normally done via LU factorization. The factorization is $PAQ = LU$, where P and Q serve two purposes: reducing fill-in and maintaining numerical accuracy. In left-looking sparse LU factorization, Q is chosen to reduce fill-in and P is chosen via partial pivoting. Solving $Ly = Pb$ and $Uz = y$ gives the permuted solution, and then $x = Qz$ gives the solution to $Ax = b$.

The `cs_lusol` function solves $Ax = b$ using LU factorization. The parameter `order` determines the ordering method used. The natural order ($Q = I$) is `order=0`. If the matrix has a mostly symmetric nonzero pattern and large enough entries on its diagonal, then a minimum degree ordering of $A + A^T$ (`order=1`) with a small `tol` is best. Otherwise, an ordering of A^TA with `tol=1` is more suitable (`order=2` removes dense rows of A prior to ordering A^TA; `order=3` does not).

```
int cs_lusol (int order, const cs *A, double *b, double tol)
{
    double *x ;
    css *S ;
    csn *N ;
    int n, ok ;
    if (!CS_CSC (A) || !b) return (0) ;     /* check inputs */
    n = A->n ;
    S = cs_sqr (order, A, 0) ;              /* ordering and symbolic analysis */
    N = cs_lu (A, S, tol) ;                 /* numeric LU factorization */
    x = cs_malloc (n, sizeof (double)) ;    /* get workspace */
    ok = (S && N && x) ;
    if (ok)
    {
        cs_ipvec (N->pinv, b, x, n) ;       /* x = b(p) */
        cs_lsolve (N->L, x) ;               /* x = L\x */
        cs_usolve (N->U, x) ;               /* x = U\x */
        cs_ipvec (S->q, x, b, n) ;          /* b(q) = x */
    }
    cs_free (x) ;
    cs_sfree (S) ;
    cs_nfree (N) ;
    return (ok) ;
}
```

8.4 Using a Dulmage–Mendelsohn decomposition

The Dulmage–Mendelsohn decomposition provides a precise characterization of the structurally overdetermined, well-determined, and underdetermined parts of a linear system $Ax = b$. It allows the LU and QR factorizations to be applied to submatrices that have structural full rank, have a zero-free diagonal, and always have the strong Hall property. This simplifies the theorems and algorithms and ensures that the symbolic analysis is as tight as possible. Consider the matrix $C = PAQ$, where P and Q are the Dulmage–Mendelsohn permutations from (7.6). Let $C_{11} = [A_{11}A_{12}]$

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

and $C_{33} = [A_{34}; A_{44}]$. The system $Cx = b$ becomes

$$\begin{bmatrix} C_{11} & C_{12} & C_{13} \\ & C_{22} & C_{23} \\ & & C_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix},$$

where $C_{22} = A_{23}$. The overdetermined system $C_{33}x_3 = b_3$ can first be solved for x_3 using a QR factorization to obtain a least squares solution. Next, $C_{22}x_2 = b_2 - C_{23}x_3$ can be solved for x_2 using an LU factorization. When solving this system the block upper triangular form of C_{22} should be exploited (see Problem 7.4). Finally, the underdetermined system $C_{11}x_1 = b_1 - C_{12}x_2 - C_{13}x_3$ can be solved for x_1 using the QR factorization of C_{11}^T.

This method is illustrated by the `cs_dmsol` M-file. It is able to find consistent solutions to rank-deficient problems, assuming the structural rank and numeric rank are equal. It finds the least squares solution if A is overdetermined, even though it relies on an LU factorization for the well-determined part of the system.

```
function x = cs_dmsol (A,b)
%CS_DMSOL x=A\b using the coarse Dulmage-Mendelsohn decomposition.
%   x = cs_dmsol(A,b) computes x=A\b where A may be rectangular and/or
%   structurally rank deficient, and b is a full vector.
%
%   See also CS_QRSOL, CS_LUSOL, CS_DMPERM, SPRANK, RANK.

[m n] = size (A) ;
[p q r s cc rr] = cs_dmperm (A) ;
C = A (p,q) ;
b = b (p) ;
x = zeros (n,1) ;
if (rr(3) <= m && cc(4) <= n)
    x (cc(4):n) = cs_qrsol (C (rr(3):m, cc(4):n), b (rr(3):m)) ;
    b (1:rr(3)-1) = b (1:rr(3)-1) - C (1:rr(3)-1, cc(4):n) * x (cc(4):n) ;
end
if (rr(2) < rr (3) && cc(3) < cc(4))
    x (cc(3):cc(4)-1) = ...
        cs_lusol (C (rr(2):rr(3)-1, cc(3):cc(4)-1), b (rr(2):rr(3)-1)) ;
    b (1:rr(2)-1) = ...
        b (1:rr(2)-1) - C (1:rr(2)-1, cc(3):cc(4)-1) * x (cc(3):cc(4)-1) ;
end
if (rr(2) > 1 && cc(3) > 1)
    x (1:cc(3)-1) = cs_qrsol (C (1:rr(2)-1, 1:cc(3)-1), b (1:rr(2)-1)) ;
end
x (q) = x ;
```

The Dulmage–Mendelsohn decomposition can also be used to exploit sparsity in x and b. Consider a graph G where $(j, i) \in E$ if the (ij)th block is nonzero; G is sometimes called the *acyclic reduction* of the graph of C. Solving $Ax = b$ is then analogous to solving the sparse upper triangular system $Ux = b$, where U is sparse and k-by-k, and where k is the number of fine blocks in the Dulmage–Mendelsohn decomposition. A depth-first traversal of this graph, starting from nonzero blocks of b, will determine the blocks of x that are nonzero.

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

8.5 MATLAB sparse backslash

Almost the entire book (and more) is encapsulated in the single MATLAB statement `x=A\b`. The single character "\" invokes a host of powerful matrix solvers (LAPACK, the BLAS, UMFPACK, CHOLMOD, GPLU, Givens-based sparse QR factorization, Maple, and many specialized solvers). The algorithm follows the following steps in order. It uses the first method that fits the given rule and succeeds. If the linear system is not too ill-conditioned (and even sometimes when it is), backslash (`mldivide` as it is formally known) nearly always finds a good solution.

1. If `A` is a symbolic matrix, the Symbolic Toolbox (Maple) solves it symbolically.

2. If `A` is sparse and diagonal, `b` is scaled. See `cs_scale` in Problem 2.4.

3. If `A` is sparse, square, and banded (with a sufficiently dense band), either a tridiagonal solver or a band solver (in LAPACK) is used.

4. If `A` is a lower or upper triangular sparse matrix, a forward solve or backsolve is used (like `cs_lsolve` and `cs_usolve`). If it is a full matrix, the BLAS triangular solvers are used.

5. If `A` is a permuted triangular matrix, a permuted backsolve is used (one solver for the sparse case and another for the full case). See Problem 3.7.

6. If `A` is symmetric (or Hermitian) and has real positive diagonal elements, a Cholesky factorization is tried. It uses CHOLMOD in the sparse case (either an up-looking algorithm much like `cs_chol` if `A` is very sparse or a supernodal algorithm otherwise). The matrix is first permuted via an approximate minimum degree ordering algorithm (see `cs_amd`). This algorithm uses most of the theory and algorithms presented in Chapters 2 through 4 and Chapter 7. In the full case, it uses LAPACK Cholesky factorization routines. This step is terminated early if the matrix is found not to be positive definite.

7. If `A` is a full Hessenberg matrix, it is reduced to an upper triangular matrix and the BLAS backsolve is used.

8. If `A` is square and sparse, UMFPACK is used to perform an LU factorization, `L*U=P*(R\A)*Q`, where `R` is a diagonal matrix that scales the rows of `A` (GPLU can be optionally used instead; it can be faster for very sparse matrices). UMFPACK selects one of three ordering strategies: COLAMD, AMD, or a cheap diagonal matching followed by AMD, depending on the matrix properties (how symmetric its nonzero pattern is and how many entries are on the diagonal). It uses its default pivot tolerances (0.001 for diagonal entries and 0.1 for off-diagonal entries). These tolerances lead to a much sparser factorization than partial pivoting but may result in a numerically unstable factorization. If $\min_i |u_{ii}| / \max_j |u_{jj}|$ is less than machine epsilon (about 2×10^{-16}), the matrix is factorized again using partial pivoting (a tolerance of 1.0). UMFPACK is a multifrontal method, based on the column elimination tree and a symbolic QR analysis. Iterative refinement with sparse backward

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

error is used [9], [135, Chap. 12]. UMFPACK relies on the theory and some of the algorithms presented in nearly the whole book.

9. If `A` is square and full, LAPACK is used.

10. If `A` is sparse and not square, a sparse QR factorization based on Givens rotations is used (Section 5.5).

11. If `A` is full and not square, a QR factorization based on Householder reflections is used (in LAPACK).

The `x=b/A` statement in MATLAB is called the forward slash, or matrix right-division (`mrdivide`). It is translated immediately into `x=(A'\b')'`, and the above algorithm for backslash is used. Type `doc mldivide` in MATLAB for more details.

Even with all its host of supporting solvers, the backslash operator in MATLAB 7.2 has its limitations. It does not attempt to use iterative methods. It makes no use of ordering methods based on graph partitioning methods, and so its fill-in can be higher than it might be otherwise. It does not use the Dulmage–Mendelsohn decomposition. It uses LU factorization for symmetric indefinite matrices, rather than methods that exploit symmetry.

Gilbert, Moler, and Schreiber [105] developed the original sparse backslash for MATLAB 4.0.

8.6 Software for solving sparse linear systems

Table 8.1 summarizes most of the available software for solving sparse linear systems via direct methods as of April 2006. The first column lists the name of the package. The next four columns describe what kinds of factorizations are available: LU, Cholesky, LDL^T for symmetric indefinite matrices, and QR. If the LDL^T factorization uses 2-by-2 block pivoting a "2" is listed; a "1" is listed otherwise [18, 19]. The next column states if complex matrices (unsymmetric, symmetric, and/or Hermitian) are supported. The ordering methods available are listed in the next four columns: minimum degree and its variants (minimum fill, column minimum degree, Markowitz, and related methods), nested or one-way dissection (including all graph-based partitionings), permutation to block triangular form, and profile/bandwidth reduction (or related) methods. The next three columns indicate what level of BLAS is used (1: vector, 2: matrix-vector, 3: matrix-matrix), if the package is parallel ("s" for shared memory or "d" for distributed memory), and whether or not the package includes an out-of-core option (where most of the factors remain on disk). Most distributed-memory packages can also be used in a shared-memory environment, since most message-passing libraries (MPI in particular) are ported to shared-memory environments. A code is listed as "sd" if it includes two versions, one for shared memory and the other for distributed memory. The next column indicates if a MATLAB interface is available. The primary method(s) used in the package are listed in the final column. Table 8.2 lists the authors of the packages, relevant papers, and where to get the code. An up-to-date table will be maintained at www.siam.org/books/fa02.

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Table 8.1. *Package features*

Package	LU	Cholesky	LDL^T	QR	Complex	Minimum degree	Nested dissection	Block triangular	Profile	BLAS	Parallel	Out-of-core	MATLAB	Method
BCSLIB-EXT	●	●	2	●	●	●	●	-	-	3	s	●	-	multifrontal
CHOLMOD	-	●	1	-	●	●	●	-	-	3	-	-	●	left-looking supernodal
CSparse	●	●	-	●	●	●	-	●	-	-	-	-	●	various
DSCPACK	-	●	1	-	-	●	●	-	-	3	d	-	-	multifrontal
GPLU	●	-	-	-	●	-	-	-	-	-	-	-	●	left-looking
KLU	●	-	-	-	●	●	-	●	-	-	-	-	●	left-looking
LDL	-	●	1	-	-	-	-	-	-	-	-	-	●	up-looking
MA27	-	●	2	-	●	●	-	-	-	-	-	-	-	multifrontal
MA28	●	-	-	-	●	●	-	●	-	-	-	-	-	right-looking Markowitz
MA32	●	-	-	-	●	-	-	-	●	1	-	●	-	frontal
MA37	●	-	-	-	●	●	-	-	-	-	-	-	-	multifrontal
MA38	●	-	-	-	●	●	-	●	-	3	-	-	-	unsymmetric multifrontal
MA41	●	-	-	-	●	●	-	-	-	3	s	-	-	multifrontal
MA42	●	-	-	-	●	-	-	-	●	3	-	●	-	frontal
HSL_MP42	●	-	-	-	-	-	●	-	●	3	d	●	-	frontal
MA46	●	-	-	-	-	-	-	-	-	3	-	-	-	finite-element multifrontal
MA47	-	●	2	-	●	●	-	-	-	3	-	-	-	multifrontal
MA48	●	-	-	-	●	●	-	●	-	3	-	-	●	left-looking
HSL_MP48	●	-	-	-	-	●	-	●	-	3	d	●	-	left-looking
MA49	-	-	-	●	-	●	-	●	-	3	s	-	-	multifrontal
MA57	-	●	2	-	●	●	●	-	-	3	-	-	●	multifrontal
MA62	-	●	-	-	●	-	-	-	●	3	-	●	-	frontal
HSL_MP62	-	●	-	-	-	-	●	-	●	3	d	●	-	frontal
MA67	-	●	2	-	-	●	-	-	-	3	-	-	-	right-looking Markowitz
Mathematica	●	●	-	-	●	●	●	●	-	3	-	-	-	various
MATLAB	●	●	-	●	●	●	-	●	●	3	-	-	●	various
Meschach	●	●	2	-	-	●	-	-	-	-	-	-	-	right-looking
MUMPS	●	●	2	-	●	●	●	-	-	3	d	-	●	multifrontal
NSPIV	●	-	-	-	-	-	-	-	-	-	-	-	-	up-looking
Oblio	-	●	2	-	●	●	●	-	-	3	-	●	-	left, right, multifrontal
PARDISO	●	●	2	-	●	●	●	-	-	3	s	-	●	left/right supernodal
PaStiX	●	●	1	-	●	●	●	-	-	3	d	-	-	left-looking supernodal
PSPASES	-	●	-	-	-	-	●	-	-	3	d	-	-	multifrontal
RF	●	-	-	-	-	-	-	-	●	-	-	-	-	product form of inverse
S+	●	-	-	-	-	-	-	-	-	3	d	-	-	right-looking supernodal
Sparse 1.4	●	-	-	-	●	●	-	-	-	-	-	-	-	right-looking Markowitz
SPARSPAK	●	●	-	●	-	●	●	-	●	-	-	-	-	left-looking
SPRSBLKLLT	-	●	-	-	-	●	-	-	-	3	-	-	-	left-looking supernodal
SPOOLES	●	●	2	●	●	●	●	-	-	-	sd	-	-	left-looking, multifrontal
SuperLU	●	-	-	-	●	●	-	-	-	2	-	-	●	left-looking supernodal
SuperLU_MT	●	-	-	-	-	●	-	-	-	2	s	-	-	left-looking supernodal
SuperLU_DIST	●	-	-	-	●	●	-	-	-	3	d	-	-	right-looking supernodal
TAUCS	●	●	1	-	●	●	●	-	-	3	s	●	-	left-looking, multifrontal
UMFPACK	●	-	-	-	●	●	-	-	-	3	-	-	●	multifrontal
WSMP	●	●	1	-	●	●	●	●	-	3	sd	-	-	multifrontal
Y12M	●	-	-	-	-	●	-	-	-	-	-	-	-	right-looking Markowitz

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Table 8.2. *Package authors, references, and availability*

Package	Authors, references	URL and/or contact
BCSLIB-EXT	Ashcraft, Grimes, Lewis, and Pierce [10, 12, 13, 167]	www.boeing.com/phantom/bcslib-ext
CHOLMOD	Davis, Hager, Chen, and Rajamanickam [30]	www.cise.ufl.edu/research/sparse
CSparse	Davis	www.cise.ufl.edu/research/sparse
DSCPACK	Heath and Raghavan [129, 130, 171]	www.cse.psu.edu/∼raghavan
GPLU	Gilbert and Peierls [109]	www.mathworks.com
KLU	Davis and Palamadai	www.cise.ufl.edu/research/sparse
LDL	Davis [29]	www.cise.ufl.edu/research/sparse
MA27	Duff and Reid [62]	www.cse.clrc.ac.uk/nag/hsl
MA28	Duff and Reid [61]	www.cse.clrc.ac.uk/nag/hsl
MA32	Duff [50]	www.cse.clrc.ac.uk/nag/hsl
MA37	Duff and Reid [63]	www.cse.clrc.ac.uk/nag/hsl
MA38	Davis and Duff [31]	www.cse.clrc.ac.uk/nag/hsl
MA41	Amestoy and Duff [3]	www.cse.clrc.ac.uk/nag/hsl
MA42	Duff and Scott [67]	www.cse.clrc.ac.uk/nag/hsl
HSL_MP42	Scott [182, 183, 184]	www.cse.clrc.ac.uk/nag/hsl
MA46	Damhaug and Reid [26]	www.cse.clrc.ac.uk/nag/hsl
MA47	Duff and Reid [64]	www.cse.clrc.ac.uk/nag/hsl
MA48	Duff and Reid [65]	www.cse.clrc.ac.uk/nag/hsl
HSL_MP48	Duff and Scott [69]	www.cse.clrc.ac.uk/nag/hsl
MA49	Amestoy, Duff, and Puglisi [6]	www.cse.clrc.ac.uk/nag/hsl
MA57	Duff [51, 66]	www.cse.clrc.ac.uk/nag/hsl
MA62	Duff and Scott [68]	www.cse.clrc.ac.uk/nag/hsl
HSL_MP62	Scott [184]	www.cse.clrc.ac.uk/nag/hsl
MA67	Reid [54]	www.cse.clrc.ac.uk/nag/hsl
Mathematica	Wolfram Research, Inc. [198]	www.wolfram.com
MATLAB	The MathWorks, Inc. [105]	www.mathworks.com
Meschach	Steward and Leyk	www.netlib.org/c/meschach
MUMPS	Amestoy, Duff, Guermouche, Koster, L'Excellent, Pralet [4, 5, 7]	www.enseeiht.fr/apo/MUMPS graal.ens-lyon.fr/MUMPS
NSPIV	Sherman [186]	www.netlib.org/toms/533
Oblio	Dobrian, Kumfert, and Pothen [42]	email: pothen@cs.odu.edu
PARDISO	Schenk, Gärtner, and Fichtner [179, 180]	www.computational.unibas.ch/cs/scicomp/software/pardiso
PaStiX	Hénon, Ramet, and Roman [132]	www.labri.fr/∼ramet/pastix
PSPASES	Joshi, Karypis, Kumar, Gupta, and Gustavson [119]	www.cs.umn.edu/∼mjoshi/pspases
RF	Neculai	www.ici.ro/camo/neculai/RF
S+	Fu, Jiao, and Yang [80, 185]	www.cs.ucsb.edu/projects/s+
Sparse 1.4	Kundert [143]	sparse.sourceforge.net
SPARSPAK	George and Liu [85, 89]	www.cs.uwaterloo.ca/∼jageorge
SPOOLES	Ashcraft and Grimes [11]	www.netlib.org/linalg/spooles
SPRSBLKLLT	Ng and Peyton [159]	email: EGNg@lbl.gov
SuperLU	Demmel, Eisenstat, Gilbert, and Li [40]	crd.lbl.gov/∼xiaoye/SuperLU
SuperLU_MT	Demmel, Gilbert, and Li [41]	crd.lbl.gov/∼xiaoye/SuperLU
SuperLU_DIST	Demmel and Li [147]	crd.lbl.gov/∼xiaoye/SuperLU
TAUCS	Chen, Rotkin, and Toledo [177]	www.tau.ac.il/∼stoledo/taucs
UMFPACK	Davis and Duff [28, 31, 32]	www.cise.ufl.edu/research/sparse
WSMP	Gupta [118, 119]	www.cs.umn.edu/∼agupta/wsmp
Y12M	Zlatev, Wasniewski, and Schaumburg [200]	www.netlib.org/y12m

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Exercises

8.1. Write a sparse backslash algorithm, just like `x=A\b` in MATLAB, that solves $Ax = b$. Assume `A` is sparse, and `b` can be a full or sparse vector. Examine `A` and determine its properties. If `A` is upper or lower triangular, use `cs_usolve` or `cs_lsolve` (or `cs_spsolve` if `b` is sparse). If it is square, symmetric, and all its diagonal entries are greater than zero, try `cs_chol`. Otherwise (or if `cs_chol` fails), use `cs_lu` if it is square or `cs_qr` if it is rectangular. Order the matrix as appropriate, factorize it, and then perform the appropriate forward/backsolves. For LU factorization, optionally select `order` and `tol` based on how symmetric the nonzero pattern is and how large the diagonal entries are relative to the off-diagonal entries. For yet more possibilities, see Sections 8.4 and 8.5. Optionally allow `b` to be a matrix.

8.2. CSparse does very little error checking of its inputs. CSparse checks only a few key error conditions: if it runs out of memory, if the matrix is singular for an LU factorization, if the matrix is not positive definite for a Cholesky factorization, if the matrix has the wrong type (compressed-column versus triplet), or if the row or column index is negative in `cs_entry`. Add more error checking to CSparse. See also Problem 2.12.

8.3. Add a floating-point operation (*flop*) counter to CSparse. Avoid adding statements such as `flop++`. Use a global `double flop` variable.

8.4. Modify `cs_qrsol` so that it returns x, r, and $\|r\|_2$.

8.5. *Iterative refinement* is a process that can improve the accuracy of the solution to $Ax = b$. In the sparse case, it is most useful in LU factorization when a small pivot tolerance is used. In MATLAB notation, `x=A\b ; x=x+A\(b-A*x)`, where of course `A` needs to be factorized only once. Add iterative refinement to `cs_lusol`. Note that `b-A*x` can be computed with `cs_gaxpy`. MATLAB uses iterative refinement with sparse backward error [9], [135, Chap. 12] in `x=A\b` when `A` is sparse and unsymmetric, so in this case iterative refinement will not lead to any improvement, since it has already been done. Compare with `cs_lu` with a very small pivot tolerance and no iterative refinement.

8.6. Modify `cs_lusol`, adding a `qrbound` parameter. Use this as the `qr` input to `cs_sqr`. For a wide range of matrices, determine how close $|L|$ and $|U|$ are to their upper bounds, computed when `qrbound=1`. Determine how good the guess is when `qrbound=0`. Add a parameter α to `cs_lusol` and `cs_sqr` that modifies the initial guess (replace $|U| = 4|A| + n$ in `cs_sqr` with α times the upper bound) and experiment with this parameter.

8.7. Modify `cs_lusol` so that `b` can have multiple columns.

8.8. Write a version `cs_lusol` where `b` is a sparse `n`-by-`k` matrix.

8.9. Repeat Problems 8.7 and 8.8 for `cs_cholsol` and `cs_qrsol`.

8.10. Repeat Problem 8.1, where `b` can have multiple columns.

8.11. Write a MATLAB interface for CXSparse. Note that MATLAB and CXSparse use different methods for storing complex values.